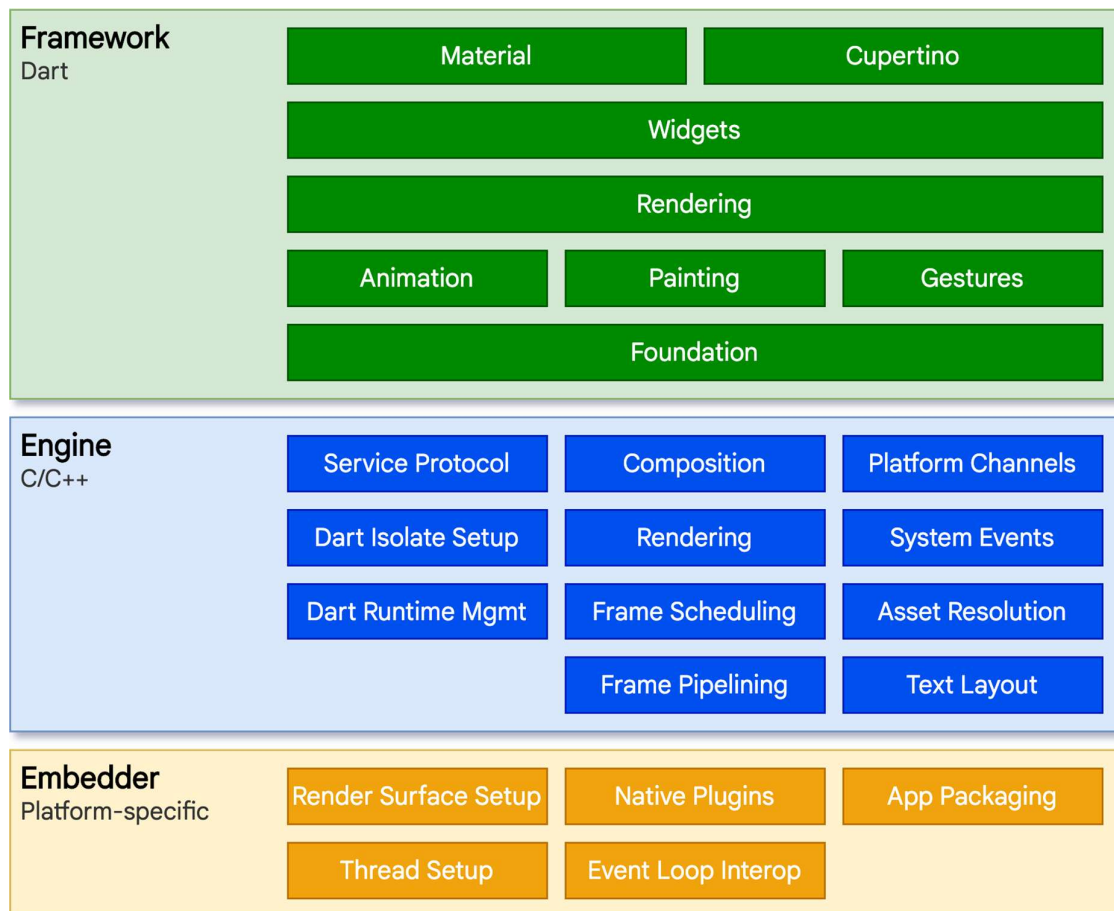


Q. Flutter Architecture

Flutter is an open-source UI framework by Google used to build natively compiled applications for mobile (Android, iOS), web, desktop, and embedded devices from a single codebase.

Its architecture is designed for high performance, fast UI rendering, and developer productivity.

Flutter follows a layered architecture with three main layers:



1. Framework Layer (Dart Layer)

This is the topmost layer and is written entirely in Dart.

It is the part that developers interact with directly.

Main Components:

(a) Material & Cupertino Widgets

- Material Widgets – For Android-style UI
- Cupertino Widgets – For iOS-style UI
- Custom Widgets – Your own reusable UI components

(b) Widgets Layer

Everything in Flutter is a widget.

Types of widgets:

- StatelessWidget – UI does not change
- StatefulWidget – UI changes based on user interaction or data

(c) Rendering Layer

- Converts widgets into render objects
- Handles layout, painting, and hit-testing
- Decides how and where UI elements appear

(d) Animation & Gestures

- Supports smooth animations
- Handles touch events like tap, swipe, drag
- Uses AnimationController, GestureDetector, etc.

(e) Foundation Library

Core utilities like:

- Logging
- Async support
- Collections
- Platform detection

2. Engine Layer (C++ Layer)

This is the core runtime of Flutter, written mainly in C++.

Responsibilities:

(a) Dart Runtime

- Executes Dart code
- Manages garbage collection
- Handles isolates (lightweight threads)

(b) Skia Graphics Engine

- Used for 2D rendering
- Draws UI directly on the screen
- Provides consistent UI across platforms

(c) Text Rendering

- Handles fonts, emojis, international text
- Uses Harfbuzz and FreeType

(d) Platform Channels

Enables communication between:

- Dart code
- Native Android (Java/Kotlin)
- Native iOS (Swift/Objective-C)

3. Platform Layer

This is the bottom layer, specific to the operating system.

Includes:

- Android OS (View system, Surface)
- iOS (UIKit, CoreGraphics)
- Windows / macOS / Linux
- Web browsers

Responsibilities:

- Access to native APIs (camera, GPS, Bluetooth)
- App lifecycle management
- Input handling
- Native plugins

Q. How Flutter Works Internally (Flow)

1. Developer writes Dart code
2. Flutter Framework builds widget tree
3. Rendering layer creates render objects
4. Engine layer uses Skia to draw pixels
5. Platform layer displays UI
6. User interaction triggers rebuild
7. Only changed widgets are redrawn

Q. Flutter Life Cycle

Flutter has two important life cycles:

1. Widget Life Cycle
2. App Life Cycle

Both are essential to understand how Flutter apps are created, updated, paused, and destroyed.

1. Widget Life Cycle (StatefulWidget)

Only StatefulWidget have a life cycle because their UI can change over time.

Widget Life Cycle Methods & Flow

1. createState()

- Called when a StatefulWidget is created
- Creates the mutable state object

2. initState()

Called once when the widget is inserted into the widget tree.

Best place to:

- Initialize variables
- Start API calls
- Set up controllers

3. didChangeDependencies()

Called after initState()

Also called when inherited widgets change

Used to:

- Access InheritedWidget
- React to theme or locale changes

4. build()

- Called every time UI needs to be redrawn
- Should return a widget
- Keep it fast and simple

5. didUpdateWidget()

Called when parent widget changes configuration

Useful to:

- Update state when widget parameters change

6. setState()

- Not a life cycle method but triggers build()
- Used to update UI

7. deactivate()

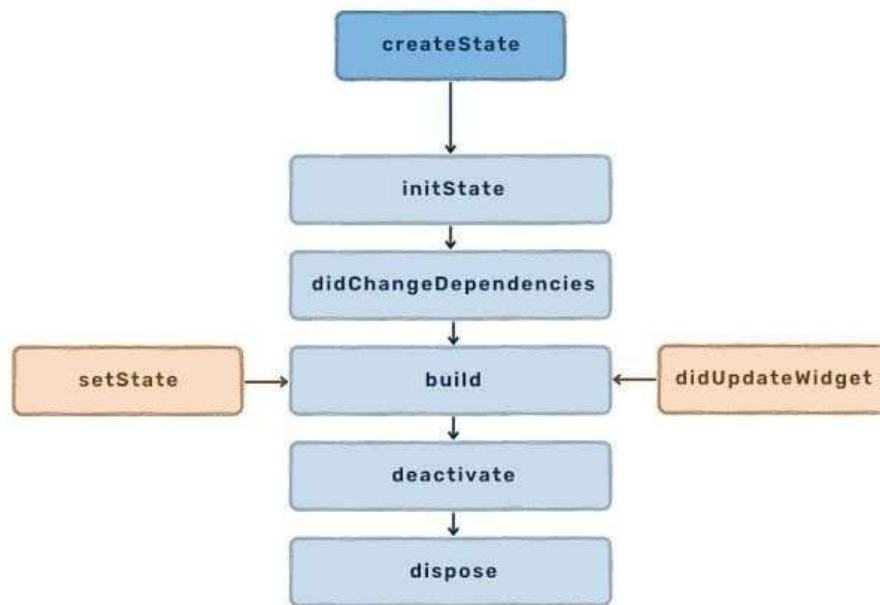
- Called when widget is removed from the widget tree temporarily
- Rarely used

8. dispose()

Called when widget is permanently removed

Best place to:

- Cancel timers
- Close streams
- Dispose controllers



2. App Life Cycle (Application Level)

This life cycle manages what happens when the entire app changes state.

Flutter observes it using `WidgetsBindingObserver`.

App Life Cycle States

State	Meaning
resumed	App is visible & active
inactive	App is inactive (call, notification)
paused	App in background
detached	App is detached from UI

FLUTTER APPLICATION LIFE CYCLE

